# CampusHubAI: Student & Admin Portal

Project Report, Architectural Details, and Presentation Guide

## 1. Executive Summary

**CampusHubAI** is a terminal-based Console Application designed to automate campus workflows for university students and administrators. Powered by a robust relational database layer, it replaces manual academic administration tasks with an interactive, secure, and fast console-based ecosystem.

Java SE 11+ | MySQL Relational Database | JDBC API | Dynamic Reconnecting Proxy | Console UI

# 2. Architecture & Design Patterns

### 🏛️ Layered Design Pattern

The project implements a clean separation of concerns using a layered pattern:

- **Model Layer:** Defines standard Java Objects (POJOs) like `Student`, `Admin`, `Attendance`, `Timetable`, etc. mapping database records.
- **Data Access Object (DAO) Layer:** Isolates the database queries (SQL inserts, updates, selects) into standalone classes, isolating business logic from data storage.
- **Service Layer:** Handles terminal user interactions and wraps the execution workflow.
- **Menu/Dashboard Layer:** Renders CLI options and processes student vs. administrator actions.

### 🔄 Reconnecting Proxy Pattern (Our Major Improvement)

Idle connections are a common pitfall in basic console database applications, as MySQL automatically drops connections that remain inactive. To solve this, we implemented a **Dynamic Proxy Connection** handler in the Database Connection layer.

> **Technical Highlight**
>
> The connection object retrieved by the DAOs is a dynamic Java reflection proxy. Every time a query is about to execute, the proxy automatically pings the server (`isClosed()` and `isValid(2)`). If the connection has dropped or timed out, it transparently closes the old socket, reconnects in the background, and runs the query, avoiding application crashes.

# 3. Database Schema Overview

The database uses relational constraints (Foreign Keys with Cascade Deletes) to ensure transaction safety and referential integrity. Below are the key tables defined in the schema:

| Table Name | Primary Key / Foreign Keys | Purpose |
|---|---|---|
| **Department** | `department_id` | Stores academic divisions (e.g. Computer Science, Mechanical). |
| **Student** | `student_id`, references `Department` | Stores credentials and contact details of college students. |
| **Admin** | `admin_id` | Stores authentication data for administrative accounts. |
| **Timetable** | `timetable_id`, references `Subject`, `Faculty` | Stores class schedules, room locations, and timeslots. |
| **Attendance** | `attendance_id`, references `Student`, `Subject` | Tracks total and present lectures per subject. |
| **Announcement** | `announcement_id`, references `Admin` | Stores general notices posted by administrators. |
| **Complaint** | `complaint_id`, references `Student` | Tracks student grievances and resolution status. |

# 4. Core Features

### 👨‍🎓 Student Dashboard

- **View Attendance:** Renders subject-wise present/total classes along with overall percentage calculations.
- **View Timetable:** Shows the daily lecture schedule. (Isolated to only show the timetable matching the student's department).
- **View Announcements:** Chronologically lists general notices.
- **Club Management:** Allows students to view active clubs and register as members.
- **Event Registration:** Displays upcoming events and records registration.
- **Notes Sharing:** View requested and approved notes shared by peers.
- **Complaints:** Submit grievances and check resolution status.

### 🛡️ Admin Dashboard

- **Manage Attendance:** Update attendance counters for students.
- **Post Announcements:** Add general notices that immediately broadcast to all students.
- **Approve Notes:** Review student notes uploads and approve or reject them.
- **Manage Complaints:** Mark student complaints as "Resolved".
- **Manage Events & Clubs:** Add/remove events and organize new clubs.

# 5. Engineering Improvements & Refactoring

During development, we resolved several critical bugs and vulnerabilities to make the system professional-grade:

### 🔒 Safe Input Parsing (No-Crash Console)

**Bug:** Using raw `Scanner.nextInt()` causes an immediate crash with an `InputMismatchException` if a user mistakenly inputs a character like "abc". Also, it leaves a trailing newline buffer that skips subsequent inputs.

**Fix:** Implemented a helper wrapper that parses lines safely. It traps the exception, clears the buffer, and prompts again without interrupting the application flow.

### 🛡️ Timetable Privacy Isolation

**Bug:** The timetable query was filtering only by semester. As a result, Computer Engineering students could see Mechanical Engineering class schedules in their dashboard.

**Fix:** Refactored the SQL query to bind the student's specific `department_id`, isolating schedules appropriately.

# 6. PowerPoint Presentation (PPT) Slide Outline

Use this structure to create a compelling presentation for your team and evaluators:

### Slide 1: Title Slide

```
Title: CampusHubAI - Academic Console Portal
Subtitle: A Secure Relational System for Student & Admin Workflow
Automation
Presented By: [Your Names]
```

### Slide 2: Problem Statement & Objectives

```
• Basic console applications crash easily on invalid input.
• Database connections automatically time out during active menu sessions.
• Manual paperwork in universities leads to data delays and redundancy.
Objective: Build a crash-proof, robust terminal interface connected to
MySQL database.
```

### Slide 3: System Architecture

```
• Layered Design Pattern: Models, Data Access Objects (DAOs), and
Services.
• Reconnecting Connection Proxy: Automatically reconnects MySQL if session
times out.
• Relational Integrity: Cascade deletes and foreign keys in database.
```

### Slide 4: Features Showcase (Student & Admin)

```
Student Features: View Attendance, Department Timetable, Submit
Complaints, Register for Events, Share Notes.
Admin Features: Post Announcements, Update Attendance, Approve Notes,
Resolve Student Complaints.
```

### Slide 5: Robustness Improvements (Demonstration Points)

- Input Validation: Clears buffer on "abc" errors without crashing.
- Dynamic Reconnection: Transparently keeps connection active.
- Department Privacy: Correctly segments timetable data.

### Slide 6: Conclusion & Future Scope

- Solid foundation for academic management.
- Future Scope: Migrate to a graphical Web Interface (React/Spring Boot), integrate automated email notifications, and implement file system storage for notes uploads.